

oblig2.cpp

```
/**
 *
 * Programmet er en kalender der man kan legge inn heldags og
 * tidsbegrensede aktiviteter på spesifikke dager.
 *
 * Hovedfunksjonalitet:
 * - Inneholder klassen 'Aktivitet' og dens to subklasser
 *   'Tidsbegrenset' og 'Heldags'. Objekter av de to siste klassene legges
 *   inn for HVER ENKELT DAG inn i to ulike vectorer inni den selvlagede
 *   containerklassen: 'Dag'
 * - De tre første klassene inneholder alle constructorer og funksjoner
 *   for å lese og skrive alle objektets data.
 * - 'Dag' inneholder en del ulike funksjoner for å håndtere datastrukturen
 *   inni seg. Det er disse medlemsfunksjonene som kalles i fra funksjonene
 *   som startes/kalles fra 'main' for EN gitt dag.
 * - Den globale vektoren 'gDagene' inneholder ALLE DE ULIKE DAGENE
 *   med hver sine ulike aktiviteter.
 *
 * @file oblig2.CPP
 * @author Kristina Jonsen
 * @date 18.02.2026
 */

#include <iostream>           // cout, cin
#include <string>              // string
#include <vector>              // vector
#include "LesData2.h"
using namespace std;         // std::cout / std::cin be gone

/*
 * Enum 'aktivitetsType' (med hva slags aktivitet dette er).
 */
enum aktivitetsType {Jobb, Fritid, Skole, ikkeAngitt};

/**
 * Baseklassen 'Aktivitet' (med navn og aktivitetstype).
 */
class Aktivitet {
private:
    string navn;
    aktivitetsType kategori;
public:
    Aktivitet() { navn = ""; kategori = ikkeAngitt; }
    void lesData();
    void skrivData() const;
};
```

```

/**
 * Subklassen 'Tidsbegrenset' (med tidspunkter for start/stopp av aktivitet).
 */
class Tidsbegrenset : public Aktivitet {
private:
    int startTime, startMin, sluttTime, sluttMin;
    bool klokkeslettOK(const int time, const int minutt) const;
public:
    Tidsbegrenset() { sluttMin = sluttTime = startTime = startMin = 0; };
    void lesData();
    void skrivData() const;
};

```

```

/**
 * Subklassen 'Heldags' (med nærmere beskrivelse av aktiviteten).
 */
class Heldags : public Aktivitet {
private:
    string beskrivelse;
public:
    Heldags() { beskrivelse = ""; };
    void lesData();
    void skrivData() const;
};

```

```

/**
 * Selvlaget container-klasse 'Dag' (med dato og ulike aktiviteter).
 */
class Dag {
private:
    int dagNr, maanedNr, aarNr;
    vector <Tidsbegrenset*> tidsbegrensedAktiviteter;
    vector <Heldags*> heldagsAktiviteter;

public:
//    Dag() { };
    Dag(const int dag, const int maaned, const int aar) {
        dagNr = dag; maanedNr = maaned; aarNr = aar; };
    ~ Dag();
    bool harDato(const int dag, const int maaned, const int aar) const;
    void nyAktivitet();
    void skrivAktiviteter() const;
    void skrivDato() const;
};

```

```

bool dagOK(const int dag, const int maaned, const int aar);
Dag* finnDag(const int dag, const int maaned, const int aar);
void frigiAllokertMemory();
void nyAktivitet();
void skrivDager(const bool inkludertAktiviteter);
void skrivEnDag();

```

```
void skrivMeny();
```

```
vector <Dag*> gDagene;          ///<  Dager med aktiviteter
```

```
/**
 * Hovedprogrammet:
 */
int main () {
    char kommando;

    skrivMeny();
    kommando = lesChar("\nKommando");

    while (kommando != 'Q') {
        switch (kommando) {
            case 'N': nyAktivitet();      break;
            case 'A': skrivDager(true);   break;
            case 'S': skrivEnDag();       break;
            default:  skrivMeny();        break;
        }
        kommando = lesChar("\nKommando");
    }

    frigiAllokertMemory();

    return 0;
}
```

```
// -----
//                               DEFINISJON AV KLASSE-FUNKSJONER:
// -----
```

```
/**
 * Leser inn ALLE klassens data.
 */
void Aktivitet::lesData() {
    cout << "\n\tNavn på aktiviteten: ";
    getline(cin, navn);

    int aktivitetstype = lesInt(
        "\n\tAktivitetstype 1 = Jobb, 2 = Fritid "
        "3 = Skole, 4 = ikke Angitt",
        1, 4);

    switch (kategori) {
        case Jobb:      cout << "Jobb"; break;
        case Fritid:    cout << "Fritid"; break;
        case Skole:     cout << "Skole"; break;
        case ikkeAngitt: cout << "Ikke angitt"; break;
    }
}
```

```

/**
 * Skriver ut ALLE klassens data.
 */
void Aktivitet::skrivData() const {
    cout << "\nAktivitet: " << navn << " er av kategori ";
    switch (kategori) {
        case Jobb:      cout << "Jobb\n";      break;
        case Fritid:     cout << "Fritid\n";     break;
        case Skole:      cout << "Skole\n";      break;
        case ikkeAngitt: cout << "Ikke angitt\n"; break;
    } cout << '\n';
}

/**
 * Leser inn ALLE klassens data, inkludert morklassens data.
 *
 * @see Aktivitet::lesData()
 * @see klokkeslettOK(...)
 */
void Tidsbegrenset::lesData() {

    Aktivitet::lesData();
    // Leser starttid til den er gyldig
    do {
        startTime = lesInt("\tStart time", 0, 23);
        startMin = lesInt("\tStart minutt", 0, 59);

        if (!klokkeslettOK(startTime, startMin))
            cout << "\n\tUlovlig starttid.\n\n";

    } while (!klokkeslettOK(startTime, startMin));

    // Leser sluttid til den er gyldig og senere enn start
    do {
        sluttTime = lesInt("\tSlutt time", 0, 23);
        sluttMin = lesInt("\tSlutt minutt", 0, 59);

        const bool ok = klokkeslettOK(sluttTime, sluttMin);
        const int startTot = startTime * 60 + startMin;
        const int sluttTot = sluttTime * 60 + sluttMin;

        if (!ok)
            cout << "\n\tUlovlig sluttid.\n\n";
        else if (sluttTot <= startTot)
            cout << "\n\tSluttid må være senere enn starttid.\n\n";
        else
            break;

    } while (true);
}

```

```

/**
 * Privat funksjon som finner ut om input er et lovlig klokkeslett.
 *
 * @param time - Timen som skal evalueres til mellom 0 og 23
 * @param minutt - Minuttet som skal evalueres til mellom 0 og 59
 * @return Om parametrene er et lovlig klokkeslett eller ei
 */
bool Tidsbegrenset::klokkeslettOK(const int time, const int minutt) const {

    return (time >= 0 && time <= 23 && minutt >= 0 && minutt <= 59);
}

/**
 * Skriver ut ALLE klassens data, inkludert morklassens data.
 *
 * @see Aktivitet::skrivData()
 */
void Tidsbegrenset::skrivData() const {          // Skriver mor-klassens data
    Aktivitet::skrivData();                      // Skriver ut felles data

    cout << "Aktiviteten starter: ";
    if((startTime <= 9) && (startMin <= 9)) {
        cout << "0" << startTime << ":" << "0" << startMin;

    } else if((startTime <= 9) && (startMin >= 10)) {
        cout << "0" << startTime << ":" << startMin;

    } else if((startTime >= 10) && (startMin >= 10)) {
        cout << startTime << ":" << startMin;

    } else cout << startTime << ":" << "0" << startMin;

    cout << '\n';

    cout << "Aktiviteten slutter: ";
    if((sluttTime <= 9) && (sluttMin <= 9)) {
        cout << "0" << sluttTime << ":" << "0" << sluttMin;

    } else if((sluttTime <= 9) && (sluttMin >= 10)) {
        cout << "0" << sluttTime << ":" << sluttMin;

    } else if((sluttTime >= 10) && (sluttMin >= 10)) {
        cout << sluttTime << ":" << sluttMin;

    } else cout << sluttTime << ":" << "0" << sluttMin;
}

/**
 * Leser inn ALLE klassens data, inkludert morklassens data.
 *
 * @see Aktivitet::lesData()

```

```

*/
void Heldags::lesData() {

    Aktivitet::lesData();
    cout << "\n\tBeskriv aktiviteten: ",
    getline(cin, beskrivelse);
}

/**
 * Skriver ut ALLE klassens data, inkludert morklassens data.
 *
 * @see Aktivitet::skrivData()
 */
void Heldags::skrivData() const {

    Aktivitet::skrivData();
    cout << "\nAktivitetsens beskrivelse: \n\t" <<this->beskrivelse;
}

/**
 * Destructor som sletter HELT begge vectorenes allokerede innhold.
 */
Dag::~Dag() {

    //Gaar igjennom vectorene og sletter elementene
    //samt alokerte minne
    for (int i = 0; i < tidsbegrensedeAktiviteter.size(); i++)
        delete tidsbegrensedeAktiviteter[i];
    tidsbegrensedeAktiviteter.clear();

    for (int i = 0; i < heldagsAktiviteter.size(); i++)
        delete heldagsAktiviteter[i];
    heldagsAktiviteter.clear();
}

/**
 * Finner ut om selv er en gitt dato eller ei.
 *
 * @param dag - Dagen som skal sjekkes om er egen dag
 * @param maaned - Måneden som skal sjekkes om er egen måned
 * @param aar - Året som skal sjekkes om er eget år
 * @return Om selv er en gitt dato (ut fra parametrene) eller ei
 */
bool Dag::harDato(const int dag, const int maaned, const int aar) const {

    return (dag == dagNr && maaned == maanedNr && aar == aarNr);
}

/**
 * Oppretter, leser og legger inn en ny aktivitet på dagen.
 *
 * @see Tidsbegrenset::lesData()

```

```

* @see Heldags::lesData()
*/
void Dag::nyAktivitet() {
    // lager pekere til vectorene
    Tidsbegrenset* tidsb;
    Heldags* heldag;

    char valg;

    valg = lesChar("\n\thvilken aktivitet skal opprettes (T/H)"
        "T = Tidsbegrenset/H = Helsdags");
    // hvis T, leses inn dataen til
    if(valg == 'T') {
        tidsb = new Tidsbegrenset;
        tidsb -> lesData();
        tidsbegrensedeAktiviteter.push_back(tidsb);

        // hvis H leser inn data til denne typen
    } else if(valg == 'H') {
        heldag = new Heldags;
        heldag -> lesData();
        heldagsAktiviteter.push_back(heldag);
    }
}

/**
 * Skriver ut ALLE aktiviteter på egen dato (og intet annet).
 */
* @see Heldags::skrivData()
* @see Tidsbegrenset::skrivData()
*/
void Dag::skrivAktiviteter() const {
    int i;

    for(i = 0; i < tidsbegrensedeAktiviteter.size(); i++) {
        cout << "\nTidsbegrenset aktiviteter denne dagen : ";
        tidsbegrensedeAktiviteter[i] -> skrivData();
    }
    for (i = 0; i < heldagsAktiviteter.size(); i++) {
        cout << "\nHeldagsaktiviteter denne dagen: ";
        heldagsAktiviteter[i] -> skrivData();
    }
}

/**
 * Skriver KUN ut egen dato.
 */
void Dag::skrivDato() const {
    cout << "Datoen for i dag: " << dagNr << '.' << maanedNr << '.' << aarNr << '\n';
}

```

```

}

// -----
//                               DEFINISJON AV ANDRE FUNKSJONER:
// -----

/**
 * Returnerer om en dato er lovlig eller ei.
 *
 * @param dag      - Dagen som skal sjekkes
 * @param maaned   - Måneden som skal sjekkes
 * @param aar      - Året som skal sjekkes
 * @return Om datoen er lovlig/OK eller ei
 */
bool dagOK(const int dag, const int maaned, const int aar) {
    if((dag <= 31) && (maaned <= 12) && (1990 <= aar <= 2030)) {
        return true;
    } else return false;
}

/**
 * Returnerer om mulig en peker til en 'Dag' med en gitt dato.
 *
 * @param dag      - Dagen som skal bli funnet
 * @param maaned   - Måneden som skal bli funnet
 * @param aar      - Året som skal bli funnet
 * @return Peker til aktuell Dag (om funnet), ellers 'nullptr'
 * @see    harDato(...)
 */
Dag* finnDag(const int dag, const int maaned, const int aar) {
    for(int i = 0; i < gDagene.size(); i++) {
        if(gDagene[i]->harDato(dag, maaned, aar) == true) {
            return gDagene[i];
        }
    }
    return nullptr;
}

/**
 * Frigir/sletter ALLE dagene og ALLE pekerne i 'gDagene'.
 */
void frigiAllokertMemory() {
    for(int i = 0; i < gDagene.size(); i++)
        delete gDagene[i];
    gDagene.clear();
}

/**
 * Legger inn en ny aktivitet på en (evt. ny) dag.
 *

```



```

* @see skrivDager(...)
* @see dagOK(...)
* @see finnDag(...)
* @see Dag::nyAktivitet()
*/
void nyAktivitet() {
    int dag,
        maaned,
        aar;

    Dag* nydag;          // Pointer til vector
                        // Registrerte dager skrives ut
    skrivDager(false);

                        // leser inn dato til aktivitet
    cout << "\nlegg inn dato aktiviteten skal holdes: ";
    dag    = lesInt("\nDag: ", 1, 31);
    maaned = lesInt("\nMåned: ", 1, 12);
    aar     = lesInt("\nÅr: ", 1990, 2030);

                        // hvis dagOK lese inn data
    if(dagOK(dag, maaned, aar) == true) {
        if(finnDag(dag, maaned, aar) == nullptr) {
            nydag = new Dag(dag, maaned, aar);
            gDagene.push_back(nydag);
            nydag->nyAktivitet();
        } else {
            finnDag(dag, maaned, aar)->nyAktivitet();
        }
    }
}

/**
 * Skriver ut ALLE dagene (MED eller UTEN deres aktiviteter).
 *
 * @param inkludertAktiviteter - Utskrift av ALLE aktivitetene også, eller ei
 * @see Dag::skrivDato()
 * @see Dag::skrivAktiviteter()
 */
void skrivDager(const bool inkludertAktiviteter) {
    if(gDagene.size() == 0){
        cout << "\n\tIngen dager er lagt til\n\n\n";
                        // Om false, skal den bare skrive dato
    } else {
        for(int i = 0; i < gDagene.size(); i++){
            if(inkludertAktiviteter == false) {
                gDagene[i] -> skrivDato();
                        // Om true, skrives dato og data ut
            } else {
                gDagene[i] -> skrivDato();
                gDagene[i] -> skrivAktiviteter();
            }
        }
    }
}

```

```
}  
}
```

```
/**  
 * Skriver ut ALLE data om EN gitt dag.  
 *  
 * @see skrivDager(...)  
 * @see dagOK(...)  
 * @see finnDag(...)  
 * @see Dag::skrivAktiviteter()  
 */  
void skrivEnDag() {  
    int dag,  
        maaned,  
        aar;  
  
    skrivDager(false);  
  
    do {  
        cout << "\nHvilken dag vil du sjekke";  
        dag = lesInt("\nDag: ", 1, 31);  
        maaned = lesInt("\nMåned: ", 1, 12);  
        aar = lesInt("\nÅr: ", 1990, 2030);  
    } while (!dagOK(dag, maaned, aar));  
  
    Dag* d = finnDag(dag, maaned, aar);  
    if (d) d -> skrivAktiviteter();  
    else cout << "\nDagen finnes ikke\n\n\n";  
}
```

```
/**  
 * Skriver programmets menyvalg/muligheter på skjermen.  
 */  
void skrivMeny() {  
    cout << "\nDisse kommandoene kan brukes:\n"  
        << "\tN - Ny aktivitet\n"  
        << "\tA - skriv ut Alle dager med aktiviteter\n"  
        << "\tS - Skriv EN gitt dag og (alle) dens aktiviteter\n"  
        << "\tQ - Quit / avslutt\n";  
}
```